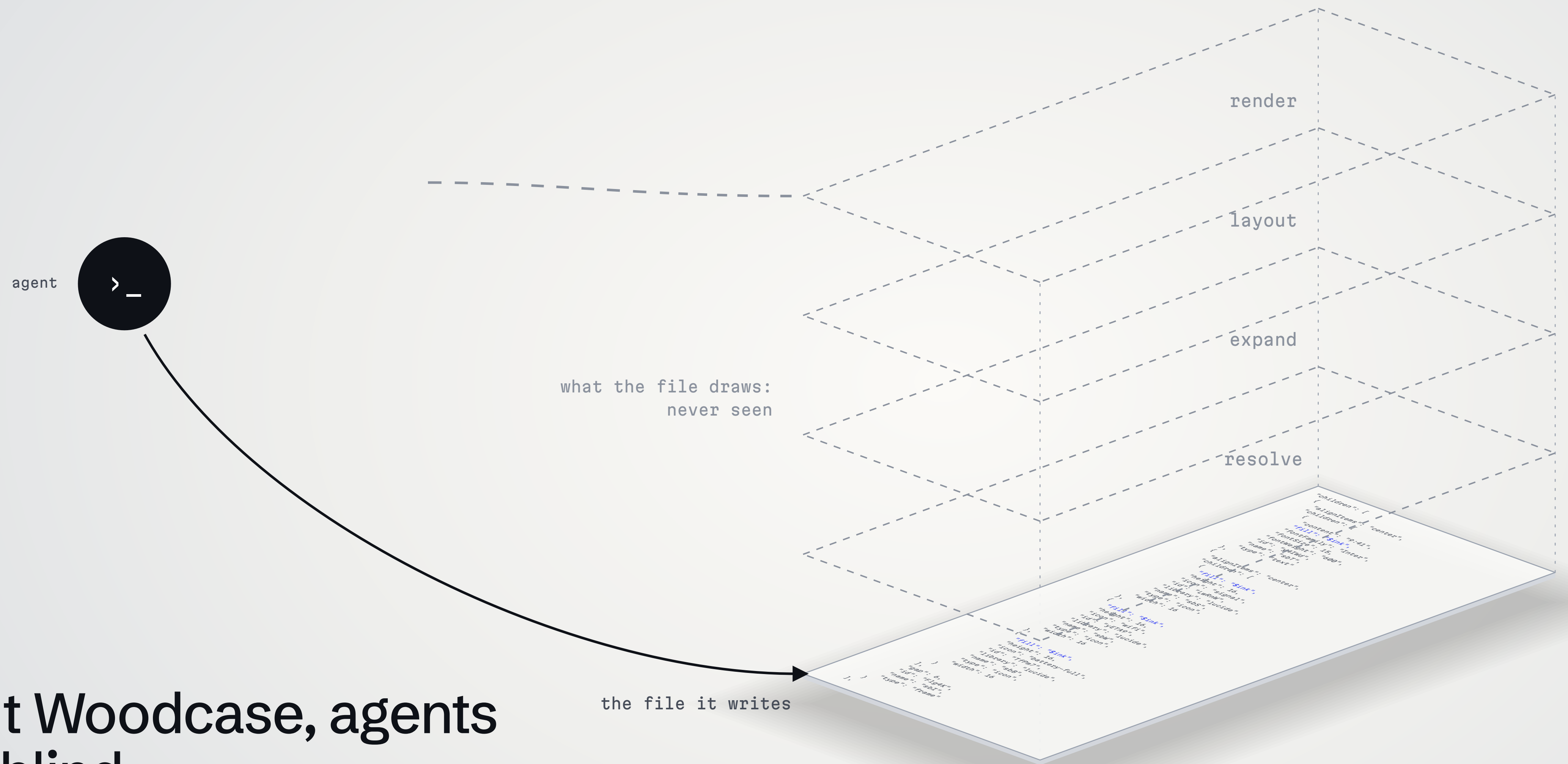


Woodcase is the design layer for agents.

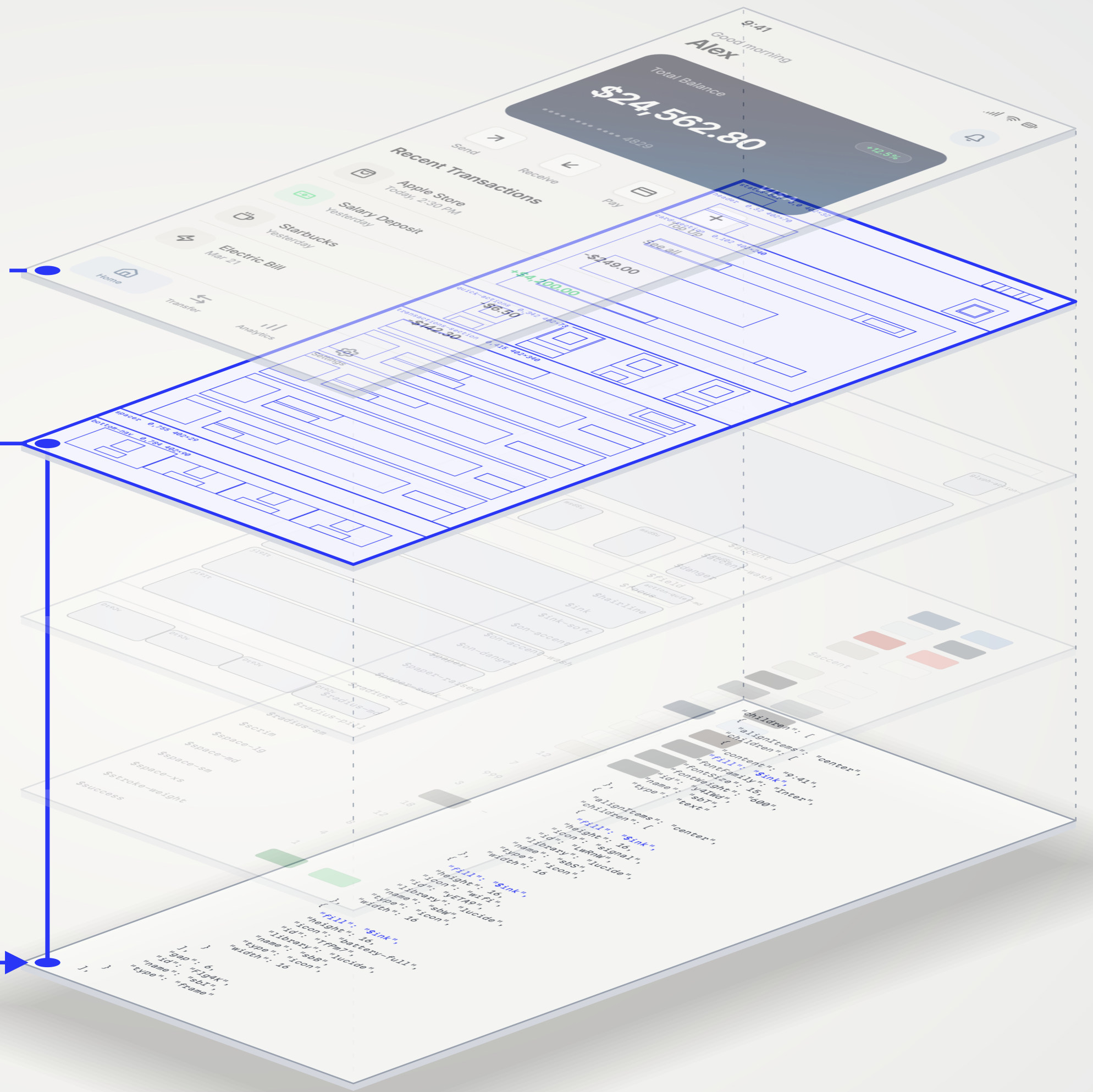
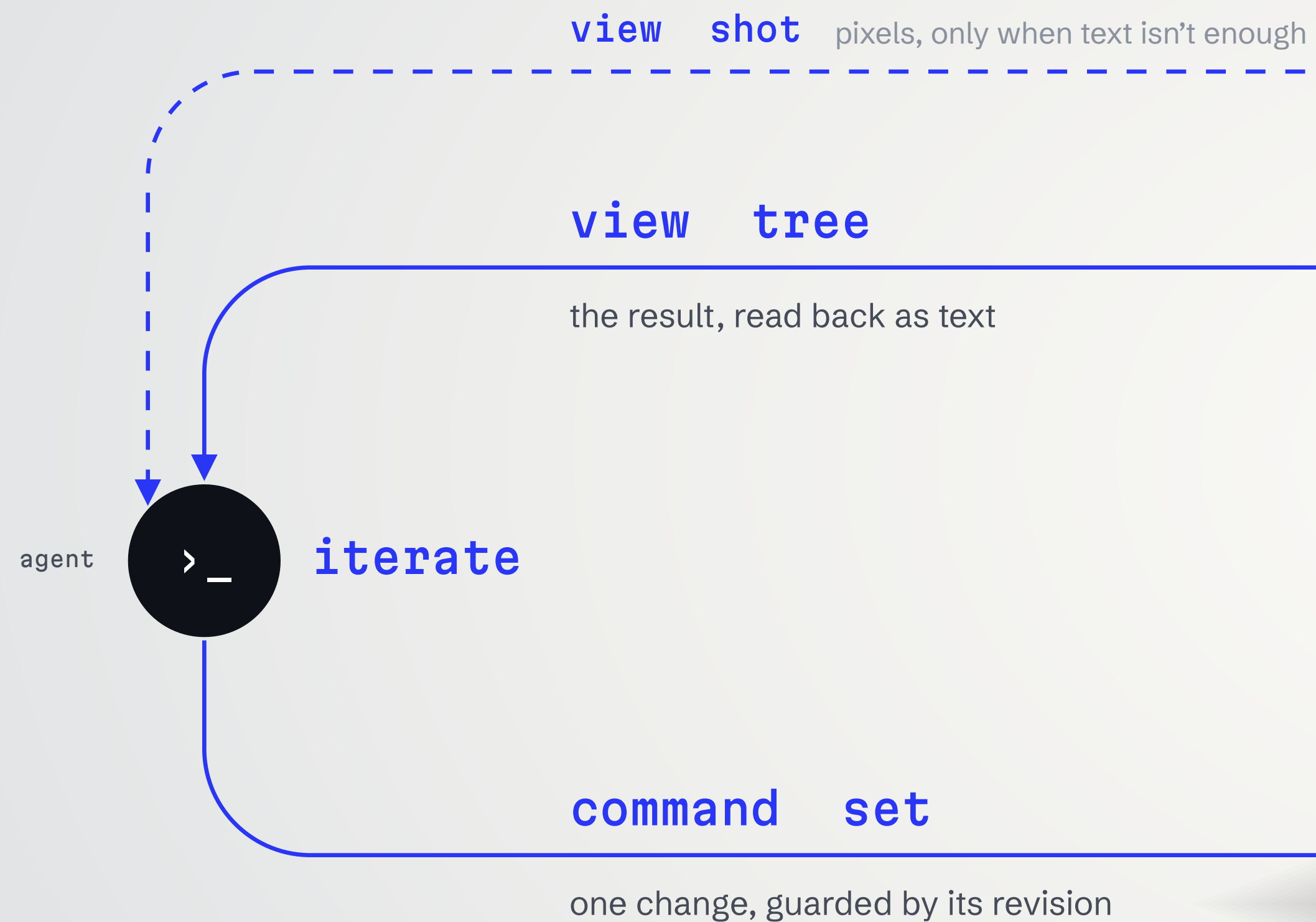


An engine for .pen files, the format of Pen, with no screen of its own. Agents run it from a terminal; apps build it in as a Swift library.



Without Woodcase, agents design blind.

An agent can write a design file, but it can't see what the file draws: not where things land, not how they look.



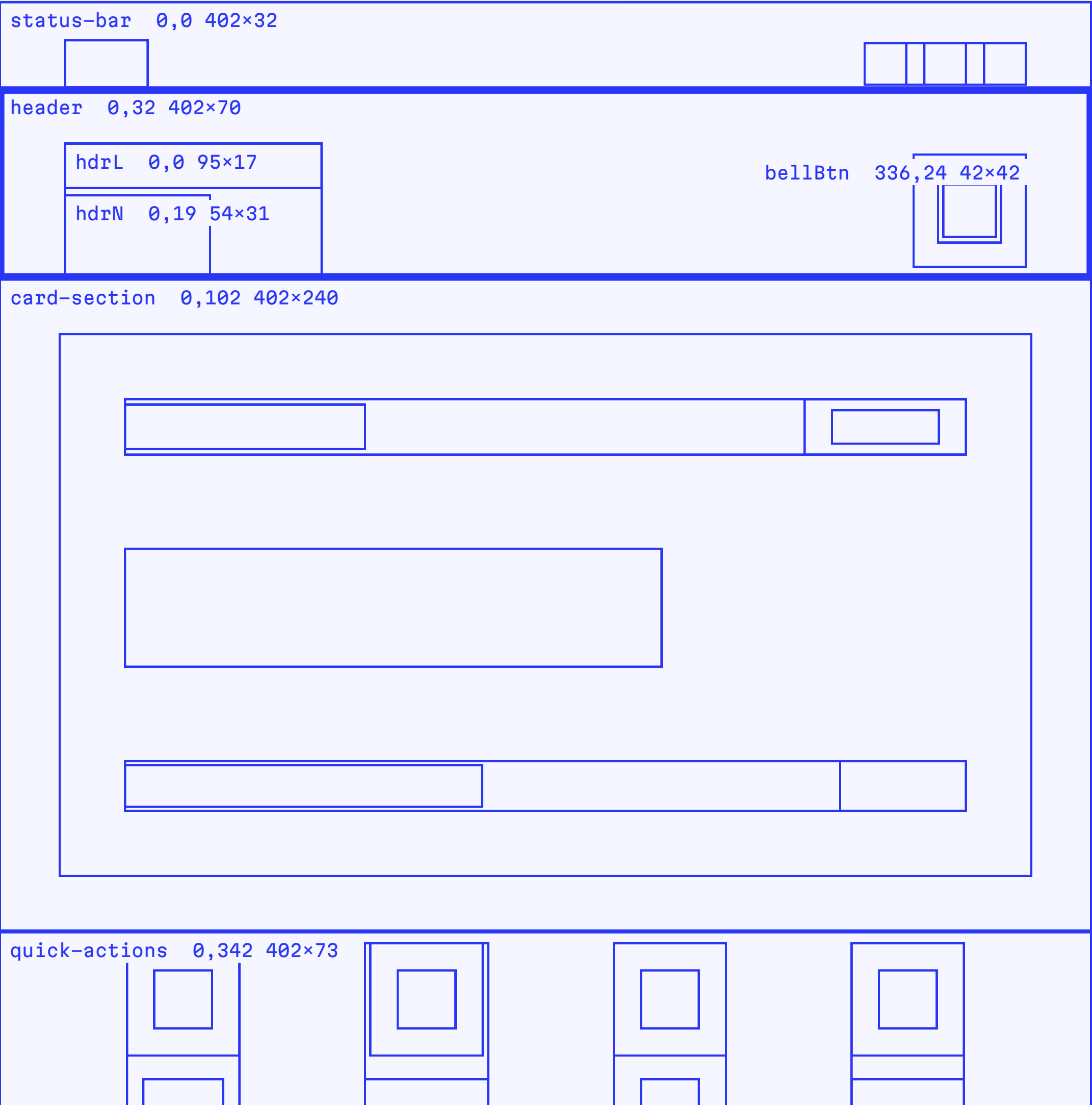
Woodcase closes the feedback loop: command, view, iterate.

Each pass is one small change and one read. Most checks need no screenshot at all.

```
$ woodcase tree banking.pen banking-home/header --props kind.content
rev b9dc00a94ca8ff89 5 rows
type  name      rect      clip  id      kind.content
frame header    0,32 402x70 6m900
frame  hdrG      24,20 95x50 CcYdb -
text   hdrL      0,0 95x17 DhDsc Good morning
text   hdrN      0,19 54x31 4YeMj Alex
ref    bellBtn +2 336,24 42x42 E97qI -
```

woodcase tree turns a layout into text an agent can read and act on.

One row per box: its type, name, position and size, and an id the next command can target.

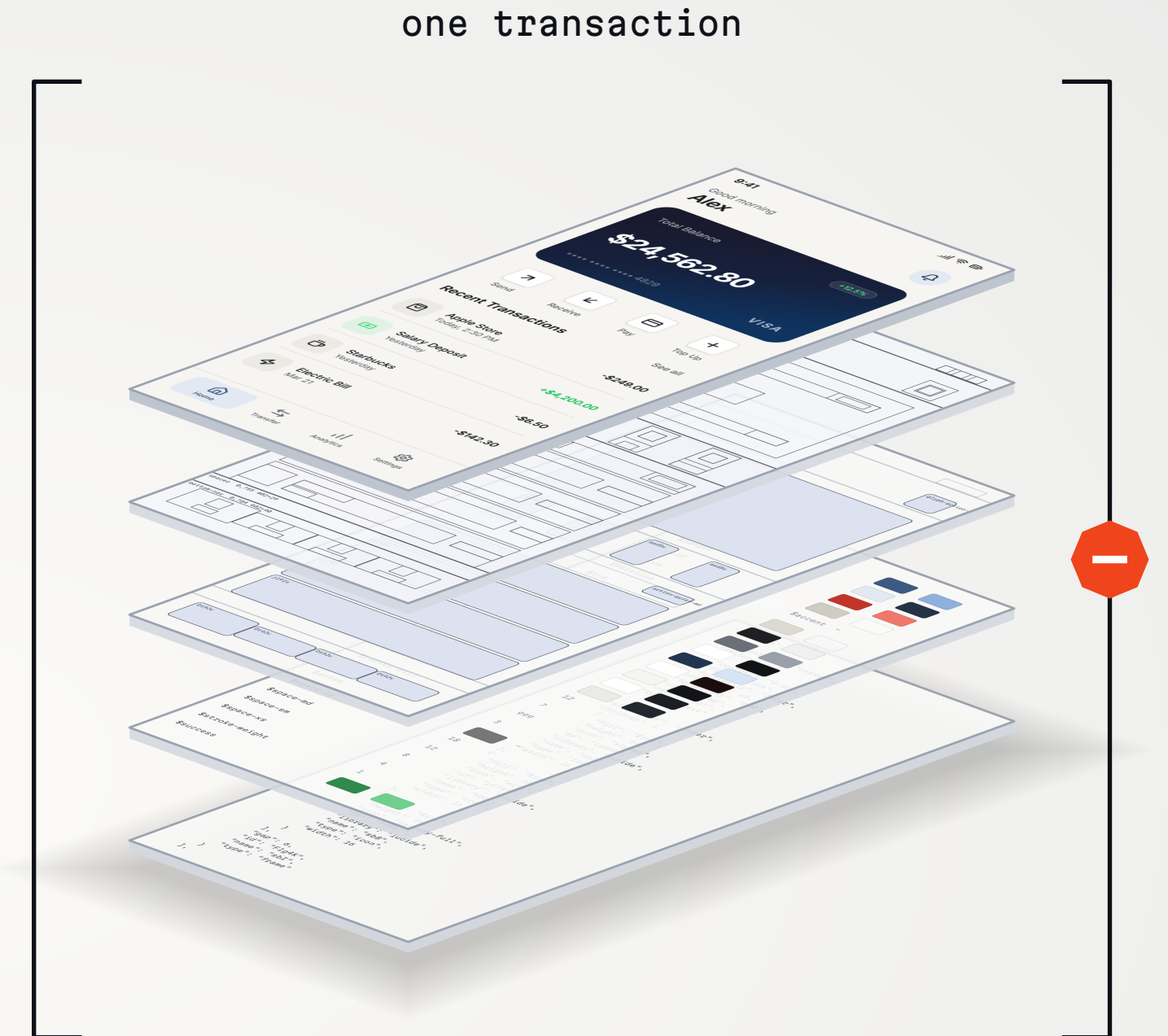


```
measure.js

// Rename a merchant, then refuse to commit if the name no longer fits.
doc.override('banking-home/transactions-section/t1/info/merchant', {
  content: 'Apple Store, Fifth Avenue, New York'
});
const clipped = doc.tree('banking-home/transactions-section/t1', { expand: true })
  .filter(r => r.clip !== 'none');
if (clipped.length) {
  const r = clipped[0];
  throw new Error(`${r.name} is ${r.rect.width}pt wide and clips its row`);
}
```

Agents think in code, so woodcase js lets them design in code.

Code is how agents already work. A script runs as one transaction, so it can check its own edit and refuse it: this one found the new name no longer fit, and nothing was saved.



```
$ woodcase js banking.pen -F measure.js

override      banking-home/transactions-section/t1  aqCub
measure.js:9:18 merchant is 242pt wide and clips its row
                throw new Error(`${r.name} is ${r.rect.width}pt wide and clips its row`);
nothing was written; banking.pen is byte for byte what it was.
[exit 1]
```

```
$ woodcase set banking.pen qa4/label kind.content=Top-up
```

qa4/label is inside the component instance banking-home/quick-actions/qa4, which stores no properties of its own –

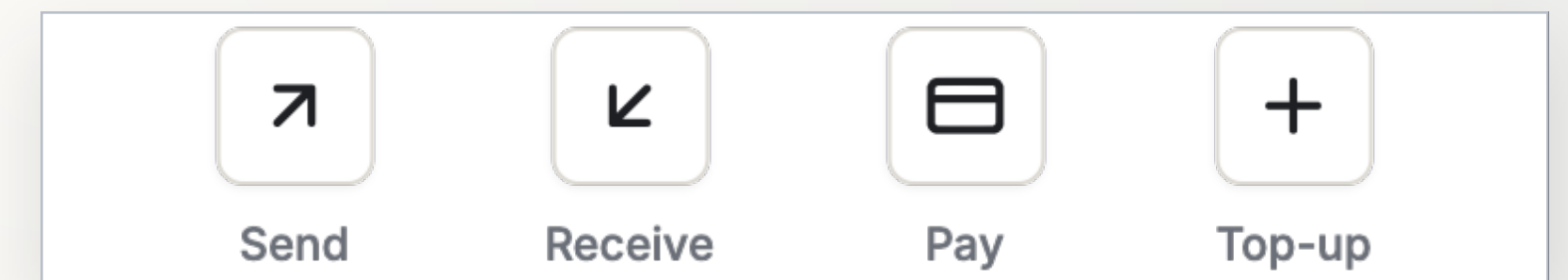
run ``woodcase override banking.pen qa4/label name=value`` instead, with raw `.pen` property names (content, not kind.content)

```
[exit 2]
```

→ `$ woodcase override banking.pen qa4/label content=Top-up`

```
banking-home/quick-actions/qa4  e0uPH  
rev  006473d00f6b4892  
document  fa131d6838e34536
```

shot, after the override



No Skill needed: agents learn Woodcase by using it.

Each refusal names the command that works, like the one above. For everything else, the tool carries its own primer:

```
woodcase help design
```



the rendered design

PNG

the render, at any scale

```
$ woodcase shot banking.pen banking-home --out home.png  
nSNTs scale=1.0 pixels=402x874 out=home.png
```

PDF

vector: one page per top-level frame, like this deck

```
$ woodcase render deck.pen --format pdf
```

CGContext

drawn by the Swift library into a context your app owns

```
let rects = PenLayoutEngine.layout(resolved)  
PenRenderer.render(expanded, layoutRects: rects, into: context)
```

React + Tailwind

98 files; this is ComponentQuickaction.tsx

```
return (  
  <div  
    className={cn("flex flex-col items-center gap-2", className)}  
    style={{
```

SwiftUI

a Swift package of 116 files; this is its QuickAction view

```
public var body: some View {  
  VStack(spacing: 8) {  
    GlyphActionOutlineLg()  
    Text("Send")  
      .penFont("Inter", size: 12, weight: 500)
```

One engine draws to PNG, PDF, your app, React and SwiftUI.

The engine is a Swift library, and the command line is one thin layer over it.

\$ TZ=UTC woodcase activity deck.pen (38 writes)

16:06:03	claude	new	-	16:06:05	claude	add	S5 Script
16:06:03	claude	add	S1 Title	16:06:05	claude	add	S5 Script/Script +1
16:06:03	claude	add	S1 Title/Stack +1	16:06:05	claude	add	S5 Script/Transaction +1
16:06:03	claude	add	S1 Title/Title +1	16:06:06	claude	add	S5 Script/Output +1
16:06:03	claude	add	S1 Title/Caption +1	16:06:06	claude	add	S5 Script/Caption +1
16:06:03	claude	add	S2 Gap	16:06:06	claude	add	S6 Users
16:06:03	claude	add	S2 Gap/Stack +1	16:06:06	claude	add	S6 Users/Refusal +1
16:06:04	claude	add	S2 Gap/Labels +1	16:06:06	claude	add	S6 Users/Fix +1
16:06:04	claude	add	S2 Gap/Paths +1	16:06:06	claude	add	S6 Users/Result +1
16:06:04	claude	add	S2 Gap/Caption +1	16:06:07	claude	add	S6 Users/Caption +1
16:06:04	claude	add	S3 Loop	16:06:07	claude	add	S7 Outlets
16:06:04	claude	add	S3 Loop/Stack +1	16:06:07	claude	add	S7 Outlets/Stack +1
16:06:04	claude	add	S3 Loop/Circuit +1	16:06:07	claude	add	S7 Outlets/Lifted +1
16:06:04	claude	add	S3 Loop/Caption +1	16:06:07	claude	add	S7 Outlets/Doors +1
16:06:04	claude	add	S4 Tree	16:06:07	claude	add	S7 Outlets/Caption +1
16:06:05	claude	add	S4 Tree/Plane +1	16:06:08	claude	add	S9 Install
16:06:05	claude	add	S4 Tree/Output +1	16:06:08	claude	add	S9 Install/Stack +1
16:06:05	claude	add	S4 Tree/Link +1	16:06:08	claude	add	S9 Install/Caption +1
16:06:05	claude	add	S4 Tree/Caption +1	16:06:08	claude	add	S9 Install/Facts +1

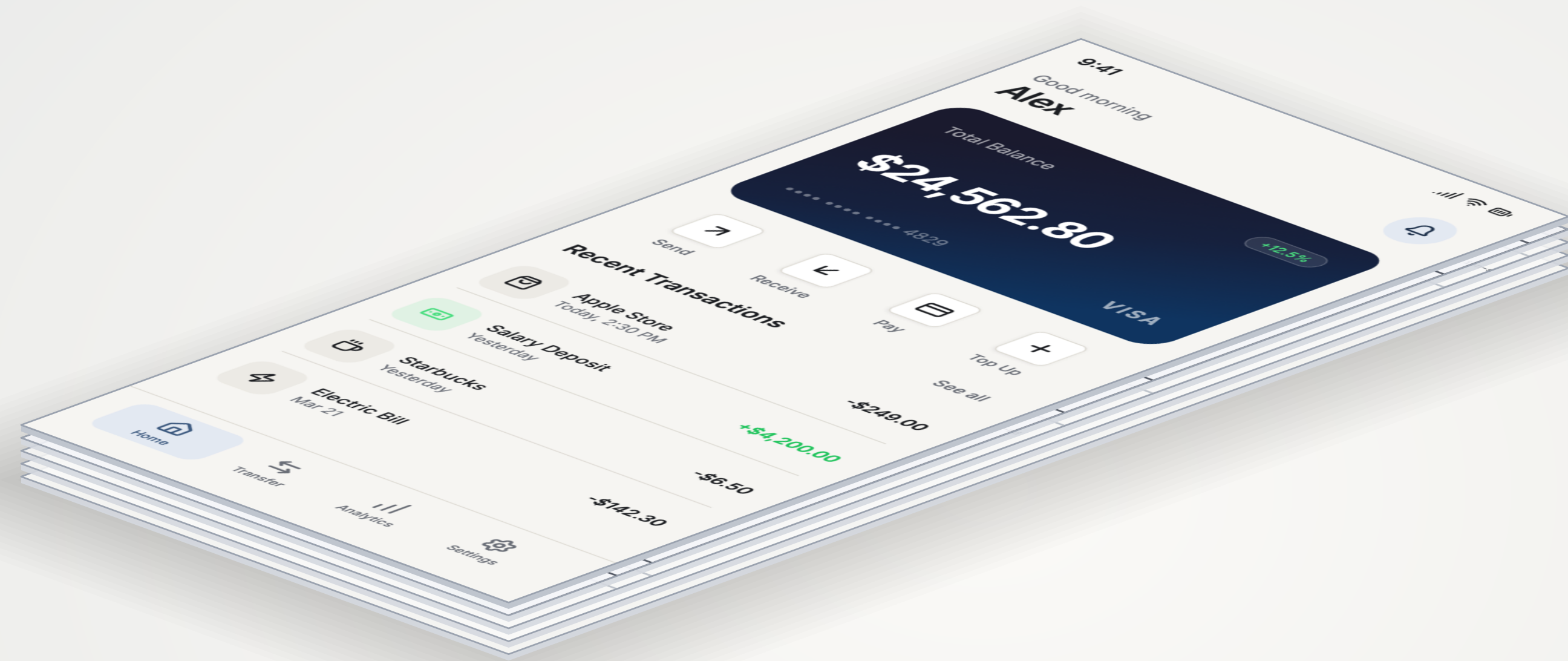
\$ woodcase undo deck.pen --dry-run

dry run nothing written
add claude 2026-09-30T16:06:08.801Z S9 Install/Facts

Of course, this deck was built with Woodcase.

Every page, and the log of every write that made it.





Install it, then tell your agent to use it.

```
$ brew install bensyverson/tap/woodcase
```

› Use woodcase to design a settings screen for my app.